Ibne Tamim
Assignment 4: Normalization
October 10, 2026

Exercise 1: Parts Order Form

A. Assumptions and Data Interpretation

The Parts Order Form contains information about customers, employees, inventory, and individual purchases. Although these attributes appear together on a single handwritten document, they describe different entities and relationships. Before normalizing the data, it is necessary to establish assumptions about how these attributes function within the business.

1. **Customer identification:** Each customer has a unique customer number, which determines the customer's name and classification. A customer may place multiple orders, but each order belongs to one customer.
2. **Part identification:** Each part number uniquely identifies a particular inventory item, including its description, category, storage location, and unit price. I assume these attributes remain consistent for a given part number.
3. **Inventory storage:** A cage code identifies a storage location rather than a specific product. Multiple parts may share the same cage, as demonstrated by the three plumbing items stored in cage G413.
4. **Employee relationships:** Employees can assist any customer, as explicitly stated in the assignment. Therefore, customer information should not be functionally dependent on the employee handling the transaction.
5. **Order identification:** Since the original form contains no order number and the assignment prohibits introducing additional attributes, I assume that the combination of order date and time uniquely identifies an order. This assumption is necessary for the proposed primary key, although a production database would benefit from a dedicated order identifier.
6. **Order line identification:** Each part appears at most once within a particular order. Consequently, the combination of order date, order time, and part number uniquely identifies an individual order line.
7. **Employee information:** Since the form provides an employee name but no employee identifier, I treat the recorded employee name as an order-level attribute rather than introducing an unsupported employee number.

These assumptions establish the functional dependencies used throughout the normalization process.

B. Normalization Process

Step 1: Unnormalized Form (UNF)

The original handwritten order form combines order-level information with a repeating collection of purchased parts.

An order contains one customer, one employee, and one date and time, but it may include several parts. Consequently, the number of part entries varies depending on the transaction.

Initial relation: PARTS_ORDER_UNF

Order attributes	Repeating part attributes
customerNumber	partNumber
customerName	partName
customerType	partType
orderDate	cageCode
orderTime	quantityOrdered
employeeName	unitPrice

The original order contains the following customer and transaction information:

Attribute	Recorded value
customerNumber	H G54587
customerName	Jeff Peterson
customerType	Consumer
orderDate	7/1/2024
orderTime	10:30 AM
employeeName	D. Harrison

The repeating group contains four items:

partNumber	partName	quantityOrdered
10654	Float Control	4
10456	Modulator	3
10776	Hose Assembly	7
10657	Float Assembly	5

The remaining part attributes are:

partNumber	partType	cageCode	unitPrice
10654	Plumbing	G413	12
10456	Electrical	H433	7
10776	Plumbing	G413	9
10657	Plumbing	G413	10

Reasoning: The problem with this structure is not simply that an order contains multiple parts. The issue is that those parts are represented as a repeating group within a single order record. This makes the structure dependent on how many items a customer purchases and prevents each part entry from being treated as an independent relational record.

The first normalization step must therefore eliminate the repeating group and represent every order-part combination separately.

Step 2: First Normal Form (1NF)

To satisfy 1NF, I transform the repeating group into individual records. Each row now represents one part purchased in one order, and every attribute contains a single atomic value.

Relation: PARTS ORDER 1NF

Primary key: (orderDate, orderTime, partNumber)

For readability, the single relation is displayed in two sections. Both sections represent the same four records and are matched by their composite keys.

Order and customer attributes

orderDate (PK)	orderTime (PK)	partNumber (PK)	customerNumber
7/1/2024	10:30	10654	H G54587
7/1/2024	10:30	10456	H G54587

7/1/2024	10:30	10776	H G54587
7/1/2024	10:30	10657	H G54587

Each record also contains customerName = Jeff Peterson, customerType = Consumer, and employeeName = D. Harrison.

Part-specific attributes

partNumber	partName	partType	cageCode	quantityOrdered	unitPrice
10654	Float Control	Plumbing	G413	4	12
10456	Modulator	Electrical	H433	3	7
10776	Hose Assembly	Plumbing	G413	7	9
10657	Float Assembly	Plumbing	G413	5	10

Reasoning: This transformation resolves the repeating-group problem, but it does not eliminate redundancy. Customer details and employee information would be duplicated for every item in an order. Similarly, a part's name, category, storage location, and price would be repeated whenever that part appears in another transaction.

This creates potential update anomalies. For example, changing a customer's name would require updating every associated order-line record. If even one record were overlooked, the database could contain conflicting information about the same customer.

Therefore, although the relation satisfies 1NF, it does not satisfy 2NF because several attributes depend on only part of the composite primary key.

Step 3: Second Normal Form (2NF)

A relation satisfies 2NF when it is in 1NF and every non-key attribute is fully functionally dependent on the entire candidate key.

The 1NF relation has a composite primary key consisting of orderDate, orderTime, and partNumber. However, its attributes do not all depend on this complete combination.

Functional dependency analysis

Determinant	Dependent attributes
orderDate, orderTime	customerNumber, customerName, customerType, employeeName
partNumber	partName, partType, cageCode, unitPrice
orderDate, orderTime, partNumber	quantityOrdered

These dependencies reveal two distinct partial dependencies.

First, customer and employee information depends on the order's date and time, regardless of which part was purchased. Second, inventory information depends on partNumber alone, regardless of the order in which the part appears.

Only quantityOrdered requires the complete composite key because the number of units purchased is specific to both the order and the part.

I eliminate these partial dependencies by decomposing the original relation into three relations.

Relation 1: ORDER_2NF

Primary key: (orderDate, orderTime)

Attribute	Example value
orderDate (PK)	7/1/2024

orderTime (PK)	10:30
customerNumber	H G54587
customerName	Jeff Peterson
customerType	Consumer
employeeName	D. Harrison

Relation 2: PART_2NF

Primary key: partNumber

partNumber (PK)	partName	partType	cageCode	unitPrice
10654	Float Control	Plumbing	G413	12
10456	Modulator	Electrical	H433	7
10776	Hose Assembly	Plumbing	G413	9
10657	Float Assembly	Plumbing	G413	10

Relation 3: ORDER_LINE_2NF

Primary key: (orderDate, orderTime, partNumber)

orderDate (PK)	orderTime (PK)	partNumber (PK)	quantityOrdered
7/1/2024	10:30	10654	4
7/1/2024	10:30	10456	3
7/1/2024	10:30	10776	7
7/1/2024	10:30	10657	5

Reasoning: This decomposition ensures that attributes describing an order are stored at the order level, attributes describing a part are stored at the inventory level, and attributes describing an order-part relationship are stored at the line-item level.

The most important distinction is that quantityOrdered cannot be stored exclusively in either ORDER or PART. An order may contain multiple parts, and the same part may be purchased in different quantities across orders. Its value therefore depends on the combination of both entities.

All three relations now satisfy 2NF. However, ORDER_2NF still contains a transitive dependency involving customer information.

Step 4: Third Normal Form (3NF)

A relation satisfies 3NF when it is in 2NF and its non-key attributes do not introduce transitive dependencies on the primary key.

The remaining dependency occurs in ORDER_2NF:

(orderDate, orderTime) → customerNumber

customerNumber → customerName, customerType

Together, these establish:

(orderDate, orderTime) → customerNumber → customerName, customerType

The customer's name and type are properties of the customer, not properties of the transaction itself. Keeping them in ORDER would continue to duplicate customer information whenever the same customer places another order.

To eliminate this transitive dependency, I create a separate CUSTOMER relation and retain customerNumber in ORDER as a foreign key.

Final 3NF relations

CUSTOMER

Attribute	Key
customerNumber	PK
customerName	
customerType	

PART

Attribute	Key
partNumber	PK
partName	
partType	
cageCode	

unitPrice	

ORDER

Attribute	Key
orderDate	PK
orderTime	PK
customerNumber	FK → CUSTOMER
employeeName	

ORDER_LINE

Attribute	Key
orderDate	PK, FK → ORDER
orderTime	PK, FK → ORDER
partNumber	PK, FK → PART
quantityOrdered	

Final relational schema

CUSTOMER(**customerNumber**, customerName, customerType)

PART(**partNumber**, partName, partType, cageCode, unitPrice)

ORDER(**orderDate**, **orderTime**, *customerNumber*, employeeName)

ORDER_LINE(***orderDate***, ***orderTime***, ***partNumber***, quantityOrdered)

The foreign key (orderDate, orderTime) in ORDER_LINE references the composite primary key of ORDER. The partNumber foreign key references PART, while customerNumber in ORDER references CUSTOMER.

C. Final Evaluation

The normalization process transforms a handwritten transaction form into four logically distinct relations. Each relation represents one type of information, and the relationships between them are preserved through foreign keys.

This design addresses three important database anomalies:

- **Update anomaly:** Customer and inventory details can be changed without modifying multiple transaction records.
- **Insertion anomaly:** New customers and inventory items can be recorded independently of orders.
- **Deletion anomaly:** Deleting an order line does not automatically remove the corresponding customer or inventory information.

The final design satisfies 3NF under the stated assumptions. It also preserves the original attributes without introducing surrogate keys, as required by the assignment.

Exercise 2: Therapist Appointments

A. Assumptions and Data Interpretation

The appointment dataset records therapists, patients, scheduled appointment times, and branch locations. The most important constraint is that therapists may work at multiple branches, but each therapist sees patients at only one branch on a particular date.

I make the following assumptions:

1. Each staffNo uniquely identifies a therapist and determines therapistName.
2. Each patNo uniquely identifies a patient and determines patName.
3. Each appointment involves one therapist and one patient.
4. A patient may attend multiple appointments on the same date, including appointments with different therapists.

5. A therapist cannot conduct two appointments simultaneously. Therefore, staffNo, appointmentDate, and appointmentTime together uniquely identify an appointment.
6. The combination of staffNo and appointmentDate determines branchNo because a therapist works at only one branch per day.
7. A branch may accommodate multiple therapists and patients. Branch numbers are treated as location identifiers, but no additional branch attributes are introduced.

B. Normalization Process

Step 1: First Normal Form (1NF)

The original dataset already satisfies 1NF because every attribute contains a single value, there are no repeating groups, and each row represents an individual appointment.

Relation: APPOINTMENT_1NF

Primary key: (staffNo, appointmentDate, appointmentTime)

staffNo (PK)	appointmentDate (PK)	appointmentTime (PK)	patNo	branchNo
S1011	9/12/2022	10:00	P100	M15
S1011	9/12/2022	12:00	P105	M15
S1024	9/12/2022	10:00	P108	Q10
S1024	9/14/2022	14:00	P108	Q10
S1032	9/14/2022	16:30	P105	M15
S1032	9/15/2022	18:00	P110	B13

The same relation also contains the following name attributes:

staffNo	therapistName	patNo	patName
S1011	Fred Smith	P100	Lily White
S1011	Fred Smith	P105	Jill Baker
S1024	Heidi Pierce	P108	Andy McKee
S1024	Heidi Pierce	P108	Andy McKee
S1032	Richard Levin	P105	Jill Baker
S1032	Richard Levin	P110	Jimmy Winter

Reasoning: Although the table satisfies 1NF, it stores information at several different levels of detail. Therapist names describe individual therapists, patient names describe individual patients, branch assignments describe a therapist's daily schedule, and appointment times describe individual consultations.

Combining these attributes in one relation causes unnecessary repetition. For example, Fred Smith's name appears in multiple records, while the branch assignment M15 is repeated for both of his appointments on September 12.

This redundancy indicates that the relation must be examined for partial dependencies before it can satisfy 2NF.

Step 2: Second Normal Form (2NF)

The primary key consists of staffNo, appointmentDate, and appointmentTime.

The relevant functional dependencies are:

Determinant	Dependent attribute
staffNo	therapistName
staffNo, appointmentDate	branchNo
patNo	patName
staffNo, appointmentDate, appointmentTime	patNo

Two partial dependencies require immediate attention.

First, therapistName depends only on staffNo. The therapist's identity does not change according to the appointment date or time. Second, branchNo depends on staffNo and appointmentDate, rather than the complete appointment key. This follows directly from the business rule that a therapist works at only one branch on a particular day. The patient name dependency is different: patName depends on patNo, which is a non-key attribute determined by the appointment key. This transitive dependency will be addressed in 3NF.

To remove the partial dependencies, I create the following relations.

THERAPIST_2NF

staffNo (PK)	therapistName
S1011	Fred Smith
S1024	Heidi Pierce
S1032	Richard Levin

THERAPIST_SCHEDULE_2NF

staffNo (PK)	appointmentDate (PK)	branchNo
S1011	9/12/2022	M15
S1024	9/12/2022	Q10
S1024	9/14/2022	Q10
S1032	9/14/2022	M15
S1032	9/15/2022	B13

APPOINTMENT_2NF

Primary key: (staffNo, appointmentDate, appointmentTime)

staffNo (PK)	appointmentDate (PK)	appointmentTime (PK)	patNo	patName
S1011	9/12/2022	10:00	P100	Lily White
S1011	9/12/2022	12:00	P105	Jill Baker
S1024	9/12/2022	10:00	P108	Andy McKee
S1024	9/14/2022	14:00	P108	Andy McKee

S1032	9/14/2022	16:30	P105	Jill Baker
S1032	9/15/2022	18:00	P110	Jimmy Winter

Reasoning: This decomposition separates therapist identity from daily branch assignments and individual appointments.

The THERAPIST_SCHEDULE relation is necessary because branch assignments are determined by a therapist and a date, not by a particular appointment time. Without this separation, scheduling additional appointments would repeatedly duplicate the same branch information.

The relations now satisfy 2NF because the partial dependencies have been removed. However, APPOINTMENT_2NF still stores patient names indirectly through patient numbers.

Step 3: Third Normal Form (3NF)

The remaining transitive dependency is:

(staffNo, appointmentDate, appointmentTime) → patNo → patName

This means patName is determined by patNo rather than directly by the appointment's primary key.

For example, Jill Baker appears in appointments with different therapists. If her name changed, updating only one appointment would create inconsistent patient records.

To eliminate this dependency, I create a PATIENT relation and retain patNo as a foreign key in APPOINTMENT.

Final 3NF relations

THERAPIST

Attribute	Key
staffNo	PK
therapistName	

PATIENT

Attribute	Key
patNo	PK
patName	

THERAPIST_SCHEDULE

Attribute	Key
staffNo	PK, FK → THERAPIST
appointmentDate	PK
branchNo	

APPOINTMENT

Attribute	Key
staffNo	PK, composite FK → THERAPIST_SCHEDULE
appointmentDate	PK, composite FK → THERAPIST_SCHEDULE

appointmentTime	PK
patNo	FK → PATIENT

Final relational schema

THERAPIST(**staffNo**, therapistName)

PATIENT(**patNo**, patName)

THERAPIST_SCHEDULE(***staffNo***, **appointmentDate**, branchNo)

APPOINTMENT(***staffNo***, ***appointmentDate***, **appointmentTime**, *patNo*)

The composite foreign key (staffNo, appointmentDate) in APPOINTMENT references THERAPIST_SCHEDULE. The staffNo attribute in THERAPIST_SCHEDULE references THERAPIST, and patNo in APPOINTMENT references PATIENT.

C. Final Evaluation

The final design separates four logically distinct concepts: therapist identity, patient identity, therapist branch assignments, and individual appointments. The most important improvement is the separation of THERAPIST_SCHEDULE from APPOINTMENT. This allows the database to represent the business rule that a therapist may work at different branches on different days while having only one branch assignment per day. The composite primary key of THERAPIST_SCHEDULE prevents conflicting branch assignments for the same therapist and date. The composite foreign key in APPOINTMENT ensures that each appointment refers to a valid therapist-date assignment. Separating PATIENT also eliminates repeated patient names and allows patient information to be maintained independently of appointment history. The resulting relations satisfy 3NF under the stated assumptions, reducing redundancy while preserving the original scheduling relationships.

Exercise 3: Event Staff

A. Assumptions and Data Interpretation

The Maid Better dataset records employees, contracts, events, and the number of hours employees work under particular contracts. The assignment specifies that each employee has a unique employee number and that each contract applies to exactly one event.

Based on these requirements, I make the following assumptions:

1. Each eNo uniquely identifies an employee and determines eName.

2. Each contractNo uniquely identifies a contract.
3. Each contract belongs to one event, although an event may have multiple contracts covering different service requirements.
4. Each eventNo uniquely identifies an event and determines eventLoc.
5. Employees may participate in multiple contracts, and each contract may involve multiple employees.
6. The hours attribute represents the total hours worked by an employee under a particular contract.
7. Since no shift date or time is provided, each employee-contract combination appears only once in the dataset.

B. Normalization Process

Step 1: First Normal Form (1NF)

The original dataset already satisfies 1NF because all attributes contain atomic values, there are no repeating groups, and each record represents one employee's participation in one contract.

Relation: EVENT_STAFF_1NF

eNo (PK)	contractNo (PK)	hours	eName	eventNo	eventLoc
1135	C1024	16	Smith J	H25	Queens
1057	C1024	24	Hocine D	H25	Queens
1068	C1025	28	White T	H4	Yonkers
1135	C1025	15	Smith J	H4	Yonkers
1135	C1026	10	Smith J	H25	Queens

Primary key: (eNo, contractNo)

Neither eNo nor contractNo can independently identify a record. An employee may work on several contracts, and a contract may employ several people. Their combination identifies a unique work assignment.

Reasoning: The relation satisfies the atomicity requirement of 1NF, but it contains repeated employee and event information.

For example, employee 1135 appears under three different contracts, causing the employee's name to be stored repeatedly. Similarly, event H25 appears under contracts C1024 and C1026, duplicating its location. These repeated values suggest that employee information and contract information depend on different parts of the composite primary key.

Step 2: Second Normal Form (2NF)

The original relation has the following functional dependencies:

Determinant	Dependent attributes
eNo	eName
contractNo	eventNo, eventLoc
eNo, contractNo	hours

The dependency eNo → eName is partial because the employee's name depends only on the employee number.

Similarly, contractNo → eventNo, eventLoc is partial because the event associated with a contract does not depend on which employee performs the work.

In contrast, hours depends on the complete composite key. The number of hours worked cannot be determined from the employee number alone or the contract number alone.

To eliminate the partial dependencies, I divide the relation into three tables.

EMPLOYEE_2NF

eNo (PK)	eName

1135	Smith J
1057	Hocine D
1068	White T

CONTRACT_2NF

contractNo (PK)	**eventNo**	**eventLoc**
C1024	H25	Queens
C1025	H4	Yonkers
C1026	H25	Queens

EMPLOYEE_CONTRACT_2NF

eNo (PK)	**contractNo (PK)**	**hours**
1135	C1024	16
1057	C1024	24
1068	C1025	28

1135	C1025	15
1135	C1026	10

Reasoning: The decomposition separates employee identity from contract information while preserving the relationship between employees and the contracts they work on.

EMPLOYEE_CONTRACT is particularly important because hours describes an assignment rather than an employee or a contract independently.

For example, employee 1135 worked 16 hours on C1024, 15 hours on C1025, and 10 hours on C1026. Storing hours in EMPLOYEE would incorrectly suggest that an employee has one fixed number of hours. Storing hours in CONTRACT would incorrectly suggest that all employees assigned to the same contract worked identical hours. Therefore, hours must remain in the associative relation identified by both eNo and contractNo. The resulting relations satisfy 2NF. However, CONTRACT_2NF still contains a transitive dependency between the contract number, event number, and event location.

Step 3: Third Normal Form (3NF)

The remaining dependency is:

contractNo → eventNo → eventLoc

Although a contract determines which event it belongs to, the event's location is determined by the event number rather than by the contract itself.

This distinction matters because multiple contracts can belong to one event.

For example, contracts C1024 and C1026 both belong to event H25 in Queens. If the event location changed, storing the location in each contract record would require multiple updates. Failing to update one record could produce contradictory information about the same event.

To remove this transitive dependency, I create an EVENT relation and retain eventNo as a foreign key in CONTRACT.

Final 3NF relations

EMPLOYEE

Attribute	**Key**

eNo	PK
eName	

EVENT

Attribute	Key
eventNo	PK
eventLoc	

CONTRACT

Attribute	Key
contractNo	PK
eventNo	FK → EVENT

EMPLOYEE_CONTRACT

Attribute	Key
eNo	PK, FK → EMPLOYEE

contractNo	PK, FK → CONTRACT
hours	

Final relational schema

EMPLOYEE(**eNo**, eName)

EVENT(**eventNo**, eventLoc)

CONTRACT(**contractNo**, *eventNo*)

EMPLOYEE_CONTRACT(***eNo***, ***contractNo***, hours)

C. Final Evaluation

The final design consists of four relations that represent employees, events, contracts, and employee work assignments independently.

Separating EVENT from CONTRACT eliminates the repeated storage of event locations. This is necessary because one event may have multiple contracts, but each contract belongs to only one event.

The EMPLOYEE_CONTRACT relation resolves the many-to-many relationship between employees and contracts. Its composite primary key ensures that each employee-contract assignment is unique, while its foreign keys preserve references to the corresponding employee and contract records.

The decomposition also addresses important data anomalies. Employee names can be updated without modifying assignment records, new contracts can be created independently of employee assignments, and deleting a completed assignment does not require deleting the employee or contract itself.

Under the stated assumptions, each non-key attribute depends on an appropriate candidate key without partial or transitive dependencies. Therefore, the final relations satisfy 3NF.

Conclusion

These three exercises demonstrate that normalization is not simply the process of dividing large tables into smaller ones. It requires identifying what each attribute represents, determining its functional dependencies, and ensuring that the resulting relations accurately reflect the underlying business rules.

The Parts Order Form required eliminating a repeating group before addressing partial and transitive dependencies. The therapist appointment and event staffing datasets already satisfied 1NF, but still contained dependencies that caused unnecessary duplication.

In each exercise, decomposition was guided by the relationship between attributes rather than by the number of columns in the original table. Primary keys were selected to uniquely identify records, while foreign keys were used to preserve relationships between the resulting relations.

The final 3NF designs reduce redundancy, improve consistency, and minimize insertion, update, and deletion anomalies while retaining the information represented in the original datasets.